دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

مبانی و کاربردهای هوش مصنوعی

پروژه اول(Sokoban)

استاد: دکتر مرضیه حسینی

دستیاران آموزشی:

مرضیه کرمی

مسیح روغنی

فاطمه صیادزاده

نیم‌سال دوم ۱۴۰۴–۱۴۰۵

پروژه اول: حل بازی سوکوبان با الگوریتم‌های جستجو

در این پروژه، مسئله‌ی بازی سوکوبان به‌عنوان یک مسئله‌ی جستجو مدل‌سازی شده است. در این بازی، هدف رساندن تمام جعبه‌ها به به خانه‌های مشخص‌شده (اهداف)، با هل دادن جعبه‌ها توسط بازیکن است. هدف اصلی این پروژه، پیاده‌سازی، تکمیل و مقایسه عملکرد الگوریتم‌های جستجوی آگاهانه و ناآگاهانه در فضای حالت و طراحی تابع heuristic به‌منظور یافتن مسیر مناسب برای حل مسئله (قرار دادن تمامی جعبه‌ها روی اهداف) می‌باشد. شما باید با استفاده از ساختار داده‌ها و محیط از پیش‌تعریف‌شده، الگوریتم‌های مختلف را در فایل مربوطه تکمیل و پیاده‌سازی کرده و درمورد تصمیم‌گیری درمورد مسیر بازیکن برای رساندن جعبه‌ها به اهداف کمک کنید. الگوریتم‌های درنظر گرفته شده برای پیاده‌سازی:

۱. BFS: Breadth-First Search

۲. UCS: Uniform Cost Search

۳. IDS: Iterative Deepening Search

٤. A*

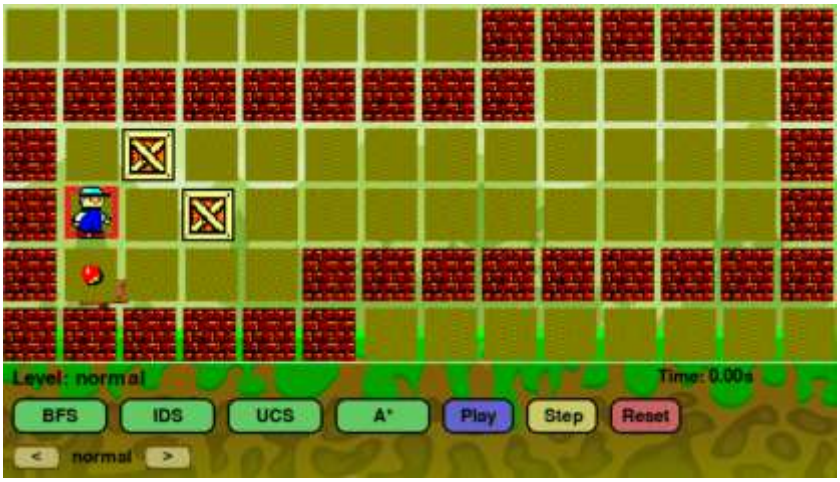
نکته: برای جستجوی با افزایش مکرر عمق می‌توانید از الگوریتم DLS (با حدس حداکثر عمق معقول) به‌عنوان جایگزین استفاده کنید.

۱. پیش‌نیازها

برای راحتی در اجرای محیط می‌توانید از Pycharm IDE استفاده کنید و نیاز به نصب کتابخانه pygame دارید.

۲. محیط

محیط بازی یک محیط شطرنجی با ابعاد N در M است که یک محیط قطعی و مشاهده‌پذیر است.



شکل ۱: نمونه‌ای از محیط بازی که باید جعبه‌ها را به مقصدها رساند

محیط بازی سوکوبان از اجزای زیر تشکیل شده است:

- دیوارها: قابل عبور نیستند.
- خانه‌های هدف: باید جعبه‌ها روی خانه‌های هدف قرار بگیرند.
- جعبه‌ها: باید به خانه‌های هدف هل داده شوند و هزینه هل دادن جعبه بیشتر از حرکت عادی است.
- بازیکن: بازیکن می‌تواند در چهار جهت بالا، پایین، چپ و راست حرکت کند. اگر در مسیر حرکت بازیکن جعبه‌ای قرار داشته باشد و خانه‌ی بعدی آن خالی باشد (بازیکن در یکی از خانه‌های محاور جعبه باشد و به سمت آن جعبه حرکت کند)، جعبه به آن سمت هل داده می‌شود.

در این پروژه، هر حرکت عادی بازیکن دارای هزینه‌ی ۱ و هر حرکت همراه با هل‌دادن جعبه دارای هزینه‌ی ۵ است. بنابراین علاوه بر پیدا کردن پاسخ، کمینه‌بودن هزینه‌ی مسیر در الگوریتم‌های آگاهانه نیز اهمیت دارد.

۳. مراحل پیاده‌سازی

برای تسهیل پیاده‌سازی جستجو، توابعی برای تحلیل و جستجو در محیط پیاده‌سازی شدند. این توابع روی اشیاء کلاس SokobanGame و State عمل می‌کنند. نحوه دسترسی به آن‌ها به صورت state/game.func(args) است و نوع خروجی آن‌ها در ادامه خواهد آمد. فهرست توابع تعامل با محیط که در اختیار شما قرار می‌گیرند:

در کلاس SokobanGame:

۱. get_grid_width() -> int: پهنای نقشه را برمی‌گرداند.

۲. get_grid_height() -> int: ارتفاع نقشه را برمی‌گرداند.

۳. get_walls() -> set: موقعیت‌های تمام دیوارهای موجود در بازی را برمی‌گرداند.

۴. get_targets() -> set: موقعیت‌های تمام خانه‌های هدف موجود در بازی را برمی‌گرداند.

۵. get_initial_state() -> state: حالت اولیه بازی شامل موقعیت بازیکن و موقعیت جعبه‌ها را بازمی‌گرداند.

۶. is_goal(state) -> bool: بررسی می‌کند که آیا تمام جعبه‌ها روی خانه‌های هدف قرار گرفته‌اند یا خیر.

۷. get_successors(state) -> list: حالت‌های ممکن بعدی را با در نظر گرفتن حرکات مجاز تولید می‌کند. خروجی شامل یک لیست از ترکیب کنش، موقعیتی که به آن منجر می‌شود و هزینه کنش خواهد بود.

۸. is_wall (position) -> bool: بررسی می‌کند که آیا موقعیت داده شده دیوار هست یا خیر.

۹. is_target(position) -> bool: بررسی می‌کند که آیا موقعیت داده شده خانه هدف هست یا خیر.

در کلاس State:

۱. get_boxes() -> list: موقعیت‌های تمام جعبه‌های موجود در حالت بازی را برمی‌گرداند.

۲. get_player_pos() -> tupple: موقعیت بازیکن در حالت بازی را برمی‌گرداند.

شما باید الگوریتم‌ها را در فایل search.py کامل کنید. تابع اکتشافی (هیوریستیک admissible) مورد استفاده در الگوریتم A* را نیز باید پیاده‌سازی کنید. در پیاده‌سازی این تابع می‌توانید از توابع تعامل با محیط که در اختیار شما قرار گرفته است استفاده کنید.

توجه کنید که توابع پیاده‌سازی شده باید بتوانند در نقشه‌های مختلف به‌کار گرفته شوند. نمونه‌هایی که می‌توانند از نظر ابعاد محیط، موقعیت اولیه بازیکن، تعداد جعبه‌ها و خانه‌های هدف متفاوت باشند.

۴. نحوه ارزیابی

هر بار پس از اجرای محیط، طول مسیر پیدا شده، تعداد گره‌های گسترش داده شده، هزینه مسیر و زمان جستجو در پنجره بازی چاپ می‌شود. ارزیابی کیفیت تابع اکتشافی پیاده‌سازی شده بر اساس تعداد گره‌های بسط داده شده و هزینه مسیر یافت‌شده توسط الگوریتم جستجوی A* خواهد بود، که در جدول زیر امتیاز مرتبط برای محیط‌های easy، normal و hard نشان داده شده است:

گره‌های بسط‌داده‌شده در نقشه ساده	گره‌های بسط‌داده‌شده در نقشه متوسط	گره‌های بسط‌داده‌شده در نقشه سخت	امتیاز
کمتر از ۲۰۰	کمتر از ۳۰۰۰	کمتر از ۵۳۰۰	نمره امتیازی
کمتر از ۲۰۰۰	کمتر از ۹۱۰۰	کمتر از ۱۹۰۰۰۰	۱۰۰٪
کمتر از ۲۴۰۰	کمتر از ۱۳۰۰۰	کمتر از ۲۳۰۰۰۰	۹۰٪
کمتر از ۲۶۰۰	کمتر از ۱۴۰۰۰	کمتر از ۳۲۲۰۰۰	۸۰٪

جدول ۱: جدول امتیازات

نکات ارائه

- لطفا نتایج خود را در قالب یک فایل pdf به همراه کدها تحویل دهید. برای قالب گزارش می‌توانید از قالب دانشکده استفاده کنید (شامل تحلیل تفاوت عملکرد الگوریتم‌های متفاوت با یکدیگر).
- پروژه را به صورت انفرادی یا تیم‌های حداکثر دو نفره می‌توانید انجام دهید.
- در زمان ارائه شفاهی باید به الگوریتم‌های پیاده‌سازی شده و بخش امتیازی مسلط باشید (در صورت انجام پروژه به صورت گروهی هر دو نفر باید به کل پروژه مسلط باشید).
- فایل‌های محیط در گیت‌هاب در اختیار شما قرار داده خواهند شد (در صورت مشکلات عدم دسترسی در گروه کلاس نیز قرار خواهد گرفت).

موفق باشید.